MERN Full Stack — Complete Roadmap 2026 (Corrected)

Sheriyans playlist verified against GitHub repos of students who completed it · Industry-Ready Goal

Errors fixed: wrong video numbers, missing RazorPay, wrong projects, misplaced Khatabook, fabricated refresh token content

WHAT WAS WRONG IN THE PREVIOUS VERSION — All 7 errors corrected

✗ **RazorPay payment integration was completely missing.** It is a full module in the course. Now added in → Phase 4. ✓

✗ **Khatabook project was placed in Phase 4 (Auth section).** It is actually an early project built during → Express/EJS — before MongoDB. Now correctly placed in Phase 2. ✓

✗ **Phase 4 checkpoint said 'E-commerce backend'.** The actual Sheriyans project is Scatch (an Instagram-like → app with image uploads and auth). Corrected. ✓

✗ **Chess.com clone (WebSockets project) was not mentioned at all.** It is one of the two major projects in the → course. Now added in Phase 4. ✓

✗ **Video numbers were wrong.** 'Videos 10-13' for all of MongoDB is too few — MongoDB spans ~8 videos. → Groupings now corrected to match actual course structure. ✓

✗ **'Refresh tokens' was listed as a topic in Phase 4.** Sheriyans does not cover refresh tokens. That was → fabricated content. Removed. ✓

✗ **JOI was listed inside the MongoDB phase.** JOI is its own separate section that comes after MongoDB. → Moved to correct position in Phase 4. ✓

ROADMAP AT A GLANCE

Part	Phases	What You Do	Weeks
Part 1 — Before Sheriyans	0–1	Git/GitHub + JS async refresh	1
Part 2 — Sheriyans Playlist	2–5	Node, Express, MongoDB, Auth, Uploads, Projects, Redis	8
Part 3 — After Sheriyans	6–10	React↔Backend, REST docs, Security, Deployment, Testing	5
Part 4 — Portfolio Project	11	Full MERN app: live, tested, documented	2–3

PART 1 — BEFORE STARTING THE SHERIYANS PLAYLIST

Phase 0 Git & GitHub — Week 1 · Days 1–3

- Install Git. Learn: init, clone, add, commit, status, log, diff.
- Branching: branch, checkout, merge. Know why branches matter.
- GitHub: create account, push repo, write README, set up .gitignore.
- .gitignore: always add node_modules/ and .env before your first commit.
- Pull requests: how they work, why teams use them.

■ **Video:** Git & GitHub Crash Course 2025 — freeCodeCamp youtube.com/watch?v=vA5TTz6BXhY

■ *Git is asked about in every developer interview. Employers look at your GitHub commit history. Do this before writing a single line of Node.js.*

■ **CHECKPOINT: Push a small project folder to GitHub with a proper README and .gitignore.**

Phase 1 JavaScript Async — Quick Refresh — Week 1 · Days 4–5

- Promises: .then(), .catch(), chaining, Promise.all().
- async/await: how it works under the hood, try/catch with async functions.
- CommonJS modules (require/module.exports) — backend default, different from React's import.
- Destructuring, spread, optional chaining — you know these from React already.

■ *The only thing genuinely new here vs React is CommonJS modules. Everything else you already know.*

■ **CHECKPOINT: Write 5 async functions: fetch fake data, handle errors, run two in parallel with Promise.all.**

PART 2 — THE SHERIYANS PLAYLIST (Confirmed topic list from course GitHub repos)

Confirmed course modules (from student GitHub repos that followed this exact playlist): JavaScript WarmUp · HTTP Module · File Handling in Node.js · Express Domination · Khatabook project · Mongoose & MongoDB · CRUD on Mongo · Advance MongoDB · Relationships in MongoDB · Referencing Method · JOI · AUTH · Multer · Production Level Setup · RazorPay · Web Sockets · Scatch project · Chess.com clone

Phase 2 — Node.js + Express Foundation

Weeks 2–3 · ~Videos 1–10

- JavaScript WarmUp: async, promises, array methods, modules — course starts here.
- Node.js core: event loop, non-blocking I/O, why it handles many requests on one thread.
- NPM: package.json, install, scripts, devDependencies vs dependencies.
- File Handling in Node.js: fs.readFile, fs.writeFile, fs.unlink — async file operations.
- HTTP Module: creating a basic server with http.createServer before using Express.
- Express Domination: routing, route params, query params, status codes, JSON responses.
- Middleware: what it is, writing custom middleware, chaining, Morgan logger.
- Cookies, sessions, CORS, cookie-parser, connect-flash — all covered in Express section.
- EJS templating + form handling — learn the concept, but React is your actual frontend.
- dotenv: store all secrets in .env — never hardcode connection strings or passwords.
- **Khatabook project:** file-based Node.js app (no MongoDB yet). Build it fully — it teaches project structure before DB comes in.

■ *Khatabook is built here — NOT later. It is a file-system project, no database. This is intentional: it teaches you to think about data before MongoDB is introduced.*

■ CHECKPOINT: Build a REST API with in-memory data: GET all, GET one, POST, PUT, DELETE. Test every route in Postman. Push to GitHub.

Phase 3 — MongoDB + Mongoose

Weeks 4–6 · ~Videos 11–18

- MongoDB basics: documents, collections, BSON. Why NoSQL for this stack.
- Mongoose: connect to Atlas, define schemas and models, connection error handling.
- CRUD on Mongo: create, find, findById, findByIdAndUpdate, findByIdAndDelete.
- Schema validation: required, unique, default, min, max, enum, custom validators.
- Advance MongoDB: all comparison operators — $eq, $ne, $gt, $gte, $lt, $lte.
- Query operators: $in, $nin, $and, $or, $exists, $regex — powerful filtering.
- Relationships in MongoDB: embedding (put data inside) vs referencing (ObjectId links).
- Referencing method + populate(): join related documents across collections.
- Separation of concerns: DB logic in models/services, not inside route handlers.

■ *The Relationships section and referencing method are two separate videos — don't skip either. Understanding when to embed vs reference is one of the most asked MongoDB interview questions.*

■ CHECKPOINT: Blog API: users, posts, comments. Posts reference author by ObjectId. GET /posts has pagination (?page=1&limit;=10) and title search using $regex. Push to GitHub.

Phase 4 Auth, JOI, Multer, RazorPay, Projects

Weeks 7–9 · ~Videos 19–29

- JOI validation: validate every incoming request body — required fields, types, formats, custom rules.
- AUTH: bcrypt for password hashing (never store plain text), JWT for token-based auth.
- Auth flow: register → bcrypt.hash → save. Login → bcrypt.compare → sign JWT → send token.
- Auth middleware: verify JWT on every protected route, attach user to req.user.
- Role-based access: admin vs regular user — middleware checks req.user.role.
- Multer: single and multiple file uploads, file type validation, size limits, disk storage config.
- Production Level Setup: professional folder structure — routes/, controllers/, models/, middlewares/, utils/, config/.
- **RazorPay integration:** payment orders, verify payment signature, webhook basics. Useful for real apps.
- WebSockets + Socket.IO: real-time bidirectional communication, rooms, events.
- **Scatch project:** Instagram-like app — user auth, image uploads with Multer, follow system, feed. The main full project.
- **Chess.com clone:** real-time multiplayer chess using Socket.IO. The WebSockets project.

■ *JOI comes before AUTH in the playlist even though the roadmap originally had it in the MongoDB section. Learn JOI first, then apply it to auth routes.*

■■ Refresh tokens are NOT covered in this course. The previous roadmap listed them — that was wrong. Sheriyans covers basic JWT only. Learn refresh tokens separately after finishing the playlist if needed.

■ **CHECKPOINT: Complete the Scatch project fully — it uses auth, Multer, MongoDB relationships, and production folder structure. Push to GitHub with a proper README.**

PART 3 — AFTER THE SHERIYANS PLAYLIST (5 gaps that make you industry-ready)

You have finished the playlist. You have a solid backend — Node, Express, MongoDB, auth, uploads, real-time with Socket.IO, and even payments with RazorPay. But your React and your backend have never talked to each other, nothing is deployed, and there are no tests. Part 3 fixes exactly that.

Phase 5 — React ↔ Backend Integration

Week 10 · Most Important Phase

- CORS: configure Express to only allow requests from your React app's domain.
- Axios in React: set baseURL, call GET/POST/PUT/DELETE endpoints from components.
- Axios interceptors: automatically attach JWT token to every outgoing request header.
- Handle 401 errors: redirect to login when token expires or is invalid.
- Store JWT: localStorage (simple, XSS risk) vs httpOnly cookie (safer) — understand both.
- Protected routes in React: check token before rendering any private page.
- VITE_API_URL environment variable: point React to your backend URL without hardcoding.
- Proxy setup in Vite config: local dev without CORS errors.
- Full flow test: register in React → data saved in MongoDB → login → protected dashboard visible.

■ **Video:** How to Create React Frontend + Express Backend — youtube.com/watch?v=yc_D8u7ETuw

■ *The Sheriyans course ends at building APIs. This phase is where you become a MERN developer — not just a backend developer. Don't skip it.*

■ CHECKPOINT: Take your Scatch backend. Build a React frontend for it: register, login, view feed, upload post. Full working flow end to end.

Phase 6 — REST API Design + Swagger Docs

Week 11

- REST naming: /api/v1/users not /getUsers — resources are nouns, methods are verbs.
- HTTP methods used correctly: GET=read, POST=create, PUT/PATCH=update, DELETE=remove.
- Status codes to know cold: 200, 201, 400, 401, 403, 404, 409, 422, 500.
- API versioning: /api/v1/ prefix on all routes — safe for future breaking changes.
- Filtering, sorting, pagination: ?page=1&limit;=10&sort;=createdAt■=desc.
- Add swagger-ui-express to one project — document every route with request/response schema.
- README: project purpose, how to run, all endpoints listed, example Postman requests.

■ **Video:** Node.js and Express.js Full Course — freeCodeCamp — youtube.com/watch?v=Oe421EPjeBE

■ *Swagger in your GitHub is a 2-hour job that makes hiring managers take your portfolio seriously.*

■ CHECKPOINT: Add Swagger docs to your Scatch backend. Every route documented. Update README with live link.

Phase 7 API Security — Week 12

- express-rate-limit: max 5 login attempts per 15 min — blocks brute-force attacks.
- Helmet.js: one import that sets 11 secure HTTP headers — always include it.
- Input sanitization: express-validator or Joi to reject injection attempts and bad data.
- Secure cookies: httpOnly flag (no JS access), secure flag (HTTPS only), sameSite.
- Safe error responses: never expose stack traces, MongoDB errors, or file paths to clients.
- Secrets management: everything in .env, .env in .gitignore, check with npm audit.

■ **Video:** API Security Fundamentals — freeCodeCamp youtube.com/watch?v=R-4_DbV1Su4

■ *Helmet + rate-limit = 2 npm packages + 5 lines of code. No excuse not to have them in every project.*

■ CHECKPOINT: Add rate limiting, Helmet, and safe error messages to your best project. Write a security checklist in the README.

Phase 8 Deployment — Make Projects Live — Week 13

- MongoDB Atlas: free cluster, whitelist 0.0.0.0/0, copy connection string.
- Render (backend): connect GitHub repo, add environment variables, deploy Express app.
- Vercel (frontend): import React repo, set VITE_API_URL to your Render URL.
- Update CORS in Express: allow your Vercel domain specifically in production.
- Test live app on your phone: register, login, upload — everything working on real URLs.
- Every GitHub project README must have the live link at the top.

■ **Video:** Deploy MERN Stack — Render + Vercel + MongoDB Atlas — 2026 youtube.com/watch?v=uGH7M2NEu3Q

■ *A live link in your portfolio is worth 10x a localhost-only project when job hunting.*

■ CHECKPOINT: Full MERN app is live: React on Vercel, Express on Render, MongoDB on Atlas. Open it on your phone.

Phase 9 API Testing — Jest + Supertest — Week 14

- Jest: write unit tests for utility functions and individual service functions.
- Supertest: fire real HTTP requests at your Express routes in tests — no browser needed.
- Test auth: register success, duplicate email (409), login success, wrong password (401).
- Test CRUD: create, read list, read one, update, delete, not found (404).
- Test authorization: protected route with no token (401), wrong role (403).
- MongoMemoryServer: tests run against fake in-memory MongoDB, never touch real data.
- Target: 15+ tests covering auth, CRUD, validation failures, and authorization.

■ **Video:** Supertest Express Tutorial — API Testing youtube.com/watch?v=T2sYitv2OAY

■ *Having any tests at all puts you ahead of most junior applicants. You don't need 100% coverage — you need variety.*

■ **CHECKPOINT: 15+ tests across your backend: 4 auth, 6 CRUD, 3 validation, 2 authorization. All passing. npm test runs clean.**

PART 4 — FINAL PORTFOLIO PROJECT (Weeks 15–17)

Build ONE project that uses everything you learned. This is what you will explain in every interview. Do NOT copy the Sheriyans Scatch project — build something original. Pick an idea that interests you. You will be talking about this project for months.

Project Idea	Why It Works for Interviews
Job Board	Auth + roles (admin/recruiter/applicant), complex filtering, real-world use case
Learning Platform	Courses, lessons, enrollment, progress tracking — complex data relationships
Expense Tracker	MongoDB aggregation for reports, categories, date filtering — shows DB skill
Appointment Booking	Time slot logic, availability, notifications — scheduling shows system design thinking
Real-time Chat	Uses Socket.IO from Sheriyans — extend what you already built

YOUR FINAL PROJECT MUST HAVE ALL OF THESE

- **Full JWT auth** — register, login, logout, role-based access (at least 2 roles)
- **MongoDB with proper schema design** — references, not everything embedded
- **File upload via Multer** — profile picture or content attachment
- **JOI or express-validator** on every POST/PUT route
- **Rate limiting + Helmet** — both added to Express app
- **RazorPay or Stripe** — payment flow if relevant to your project idea
- **React frontend connected** — Axios, interceptors, protected routes, full auth flow
- **15+ API tests** — auth, CRUD, validation, authorization
- **Swagger documentation** — every route documented
- **Deployed and live** — React on Vercel, Express on Render, MongoDB on Atlas
- **Clean README**: setup steps, architecture, API docs link, live URL, test command

WEEKLY ROUTINE

Day	What You Do	Time
Mon–Thu	Watch tutorial, code along, build the feature yourself	90–120 min
Friday	Rebuild that week's concept from scratch without watching	60–90 min
Saturday	Add tests, documentation, clean commits, push to GitHub	60 min
Sunday	Review what you built, write notes, plan next week	30 min

INTERVIEW PREP — What You Must Be Able to Explain

→ **Event loop** — how Node.js handles many concurrent requests on one thread

→ **Middleware** — write one from scratch on a whiteboard

→ **JWT** — signing, verifying, expiry, what happens when token is stolen

→ **bcrypt** — why hashing, what salt rounds do, why you can't unhash

→ **MongoDB vs SQL** — when would you choose each and why

→ **Embedding vs referencing in MongoDB** — when to use each with real examples

→ **CORS** — what it is, why it blocks requests, how to configure it

→ **Socket.IO** — how real-time works, rooms, events — you built Chess.com clone

→ **Your project** — explain every schema design decision and why

→ **Git** — branch, merge, PR — can you do this live without Googling

Bottom Line: The Sheriyans course is stronger than most free courses — it covers Redis, WebSockets, RazorPay, and two real projects (Scatch + Chess.com clone). Finish all 29 videos. Build both projects fully. Then do Part 3 in order: integrate React, add security, deploy, and add tests. One live, tested, documented project beats ten localhost-only ones in any job search.